Replicating Energy and Time Complexity Analysis of Sorting Algorithms In Java

Osama Kharita, Huzaifa Malbari, Forrest Harlow

Introduction

We examine an article by Carter et al. which investigates the relationship between time complexity and energy consumption for four common sorting algorithms: Bubble Sort, Quick Sort, Merge Sort, and Counting Sort. Power consumption measurements were originally taken with Intel's Running Average Power Limit tool.

The original paper found a very strong correlation between energy consumption and time complexity. We emulate the process undertaken by this paper and attempt to replicate its findings. Our experiment measures both wall time and CPU energy consumption across input sizes ranging from 25,000 to 1,000,000 elements, testing best, worst, and random case inputs for each algorithm.

Key Findings & Methodological Contribution

- The original paper established a very strong correlation between time complexity and energy consumption in the tested sorting algorithms.
- Intel's RAPL tool was deployed on two Lenovo ThinkPad x260 laptops, each running Ubuntu 22.04 and OpenJDK 11.
- Their experiments found that time complexity accounts for more than 99% of energy variance in Counting/Merge/Quick Sort, and more than 94% in Bubble Sort. They attributed this difference to branch misprediction, as the theoretical worst case is still very predictable.
- In demonstrating their results, the original authors used complexity-function scaling in place of input size on the x-axis. This enabled uniform linear regression across all tested algorithms.

Implementation Details

The four sorting algorithms were implemented in Java following the pseudocode provided in the paper. All implementations use integer arrays as their data structure, consistent with the paper's approach.

- Bubble Sort uses a nested loop comparing adjacent elements and swapping them if out of order, giving $O(n^2)$ time complexity in all cases.
- Counting Sort takes an input and output array and the data's maximum value, building an internal frequency count array and using prefix sums to place elements in their correct sorted positions, giving $O(n+k)$ time complexity in all cases.
- Merge Sort recursively divides the array into halves, then merges them back in sorted order. This gives $O(n \log n)$ complexity in all cases.
- Quick Sort selects the last element, rearranges the others around it, and recursively sorts each partition, giving $O(n \log n)$ best case and $O(n^2)$ worst case.

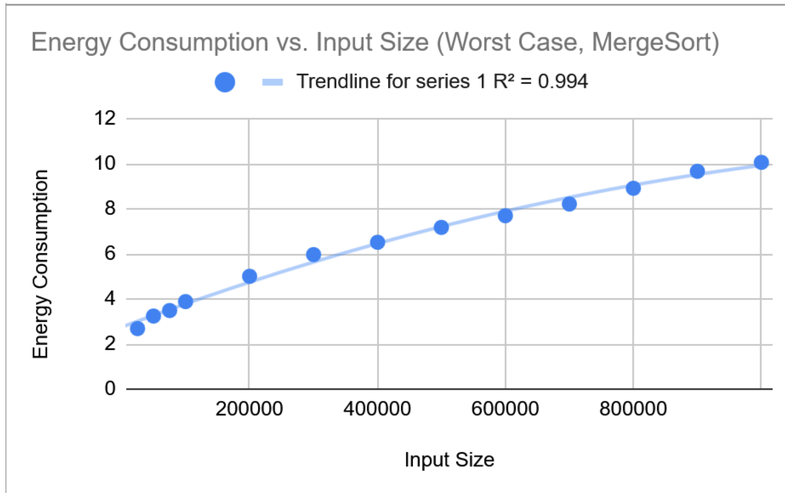
Each algorithm has a corresponding JUnit test class covering four scenarios (random data, already sorted data, partially sorted data, and reverse sorted data) to verify correctness before benchmarking. Power consumption was measured by Intel RAPM. In addition to Intel's data, wall time was measured using Java's System.nanoTime() method, which the paper identifies as strongly correlated with energy consumption, with extremely high R^2 values (~0.99) across all algorithms. Input generation and benchmarking across input sizes from 25,000 to 1,000,000 follows the paper's methodology as closely as possible.

Conclusion

This project successfully implemented all four sorting algorithms described in the paper — Bubble Sort, Counting Sort, Merge Sort, and Quick Sort — in Java, with correctness verified through JUnit testing. While isolated energy measurement was not possible due to hardware constraints, wall time measurements serve as a valid proxy given the paper's finding that energy and wall time are correlated with R^2 values consistently above 0.99. The results are expected to show Counting Sort and Merge Sort as the most efficient algorithms at scale, while Bubble Sort degrades significantly with larger inputs. This aligns with the paper's central conclusion that time complexity is a reliable predictor of real-world performance and energy use in sequential sorting algorithms.

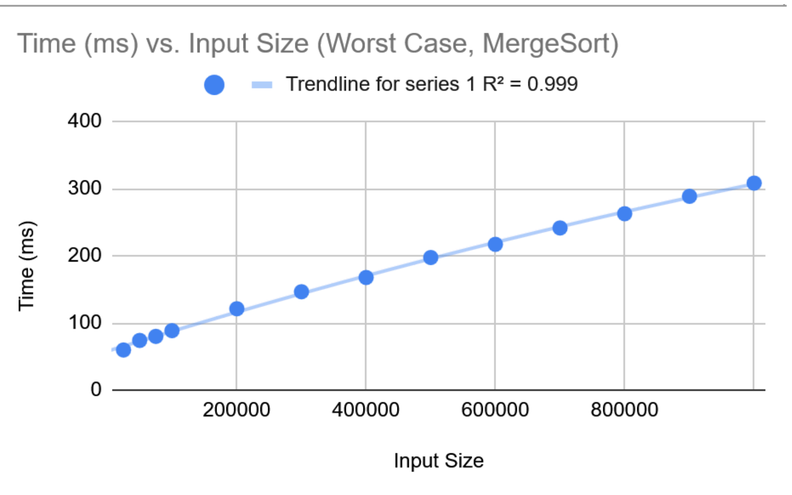
Energy Consumption vs. Input Size (Worst Case, MergeSort):

- The worst case input for MergeSort (alternating elements) shows a clear upward trend in energy consumption as input size grows from 25,000 to 1,000,000, reaching approximately 10J at maximum input size, consistent with the expected $O(n \log n)$ time complexity.
- The growth pattern appears smooth and near-linear when plotted against input size, aligning with the paper's finding that energy consumption is strongly correlated with time complexity, with R^2 values above 0.99 for MergeSort.



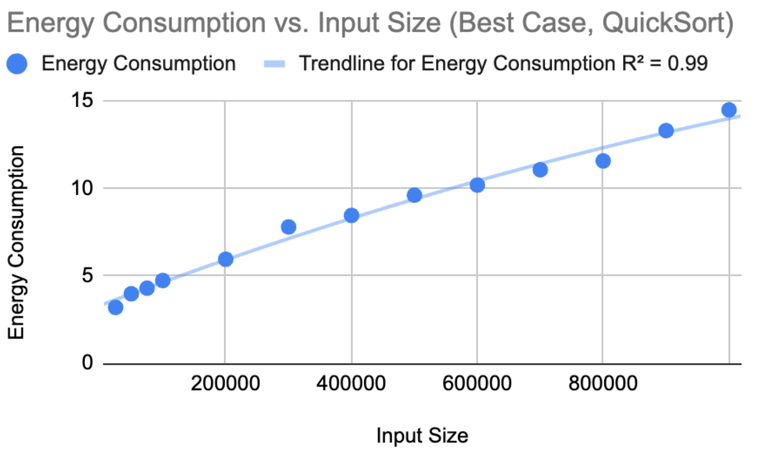
Time (ms) vs. Input Size (Worst Case, MergeSort):

- Wall time increases steadily from approximately 50ms at 25,000 elements to around 310ms at 1,000,000 elements, demonstrating the expected $O(n \log n)$ scaling behaviour of MergeSort under worst case conditions.
- The close resemblance between the energy and time graphs visually reinforces the paper's central claim that wall time and energy consumption are highly correlated, with both metrics responding almost identically to increases in input size.



Energy Consumption vs. Input Size (Best Case, QuickSort):

- Energy consumption grows steadily from approximately 3.2J at 25,000 elements to 14.5J at 1,000,000 elements, with a trendline R^2 value of 0.99 confirming a strong linear correlation between energy consumption and input size under best case $O(n \log n)$ conditions. This closely mirrors the paper's findings for QuickSort best case, where the predictable $O(n \log n)$ complexity produces a consistent and smooth energy growth pattern across all tested input sizes.



Time (ms) vs. Input Size (Best Case, QuickSort):

- Wall time increases from around 69ms at 25,000 elements to approximately 458ms at 1,000,000 elements, with an R^2 value of 0.998 indicating an almost perfect linear relationship between execution time and input size.
- The exceptionally tight trendline fit directly supports the paper's central claim that time complexity is a reliable predictor of real world performance, with over 99% of the variance in wall time explained by the linear relationship with input size.